

YAZILIM MÜHENDİLİĞİ

GİRİŞ

Ders Kaynakları :

1. Yazılım Mühendisliği, *Ian Sommerville*, 10. Basımdan Çeviri. Çeviri Editörü, Prof. Dr. N. Yasemin Topaloğlu
2. Software Engineering, *Ian Sommerville*, 9th edition, Addison-Wesley
3. Software Engineering ☐ Practitioners ☐ Approach, Roger S. Pressman.
4. Ders Notları

Değerlendirme

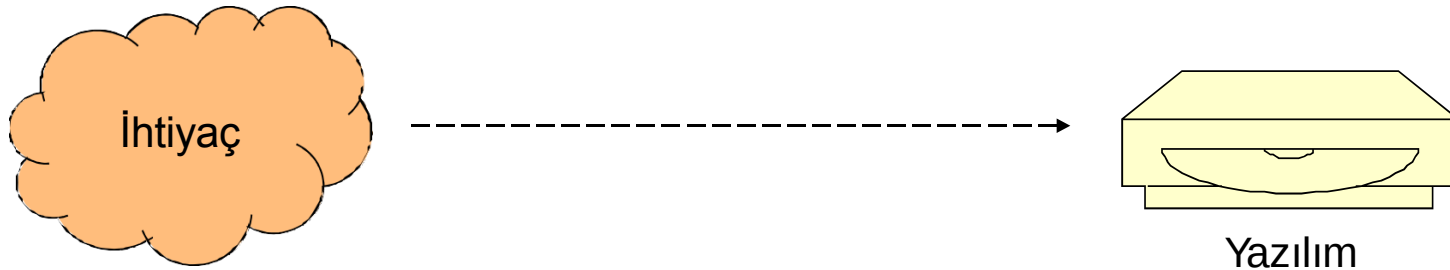
- %30 vize %20 Proje %50 Final

İçerik

- Yazılım Mühendisliği
 - ▶ Tanımlar, tarihçe, kapsam

Yazılım Nedir?

- Bilgisayar programları ile bunlarla ilişkili yapılandırma ve belgelerin bütünüdür.
 - ▶ Genel yazılım
 - Özellikleri pazardaki genel ihtiyaca göre belirlenerek geliştirilir.
 - “Generic software”, “commercial-off-the-shelf (COTS) software”
 - Örnek: MS Office yazılımı
 - ▶ Müşteriye özel yazılım
 - Özellikleri belirli bir müşterinin ihtiyacına göre belirlenerek geliştirilir.
 - “Custom software”
 - Örnek: Hava trafiği kontrol yazılımı



Yazılım

- Software
- Günümüz bilgisayar teknolojileri iki temel teoriye dayanır
 - Von-Neumann
 - Turing Makineleri

Yazılım Mühendisliği Açısından Yazılım

- Yazılımı talep eden kişiyi doğru anlamak
İsteklerin zaman içinde değişebileceğinin farkında olmak
- Problemlerin ve yeni eklentilerin yönetimini yapabilmek
- Maliyeti düşürebilmek
- Yazılım ekibini doğru şekilde kurabilmek
Kullanılacak donanımı belirlemek

Yazılım

- Eskiden “yazılım geliştirme yaşam döngüsü” Şimdi “sistem geliştirme”
 - Analiz, Kod, Raporlama, Test ...
- Yazılım denince sadece kod değil tüm sistem anlaşılmalıdır

Yazılım Geliştirme

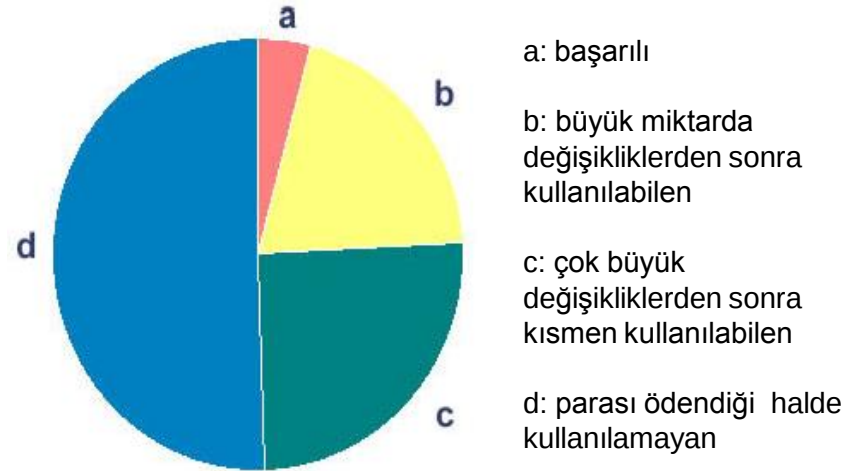
- 1940'lar:
 - ▶ Elle yazılan basit makine kodları
- 1950'ler:
 - ▶ Üretkenlik ve kaliteyi arttırmak için yazılan makro birleştiriciler ve yorumlayıcılar
 - ▶ İlk kuşak derleyiciler
- 1960'lar:
 - ▶ Üretkenlik ve kaliteyi arttırmak için yazılan ikinci kuşak derleyiciler
 - ▶ Yazılım Mühendisliği NATO Konferansı, 1968: "Yazılım mühendisliği" kavramının tartışılması
 - ▶ İlk büyük yazılım projeleri (1000 programcı)
 - ▶ Büyük iş alanları için ana-bilgisayarlar ve ticari yazılımlar
- 1970'ler:
 - ▶ UNIX ve kod kütüphaneleri için araçlar
 - ▶ Mini-bilgisayarlar ve küçük iş alanları için yazılımlar
- 1980'ler:
 - ▶ Kişisel bilgisayarlar ve iş istasyonları
 - ▶ Ticari yazılımlar
- 1990'lar:
 - ▶ Avuç-ichi bilgisayarlar ve web teknolojileri
 - ▶ Teknolojideki gelişmeler sebebiyle düşen fiyatlar ve karmaşıklaşan iş talepleri
- 2000'ler:
 - ▶ Globalleşme sebebiyle artan talepler
 - ▶ Bütünleşik geliştirme ortamları

Yazılım Krizi

- 1960'lardan itibaren yazılım ürünlerine artan talepler karşısında otomasyonu da içeren değişik çözümler uygulandıysa da yeterli üretim kapasitesine erişilemedi. Bu yetersizlik, “yazılım krizi” söylemiyle dünya literatüründe yerini aldı.
- Yazılım krizinin temel sebebi, üstel olarak artan talebe karşılık doğrusal hızla artış gösteren üretim kapasitesidir. Diğer yandan yazılım projelerindeki başarısızlık oranı da şaşırtıcı derecede yüksek olup bu krizi beslemektedir.



Yazılım ürününe arz ve talep artışları



Toplam maliyete göre yazılım başarısı

Yazılım Ürünlerine Talep

- Yazılım ürünlerine olan talep ve yazılım ürünlerinden beklentiler çok hızlı artıyor
 - ▶ Boeing 777
 - A.B.D. ve Japonya'da 1700 iş istasyonu
 - 4,000,000 Kod Satırı
 - “Kanatları olan yazılım”
 - ▶ Beyaz eşyalar, cep telefonları, otomobiller
 - ▶ Akıllı ev ve ofis sistemleri
 - ▶ ...

Bazı Hikayeler



■ Denver havaalanı otomatik bagaj sistemi

- Açılış 2 yıl gecikti
- \$27 milyon maliyet aşımı
- \$360 milyon gecikme maliyeti

■ Hava trafik kontrol (FAA modernizasyon)

- \$5.6 milyar maliyet aşımı
- 8 yıl gecikme
- 4 sistemden 2 tanesi iptal edildi, üçüncü sistemin gereksinimlerinin %52'si karşılandı

■ Comanche Helikopteri

- 10 yılda maliyeti 10 kat arttı, \$34.4 milyon
- Gereksinimleri %74 azaltıldı

Ürün Büyüklüğü Başarı İlişkisi

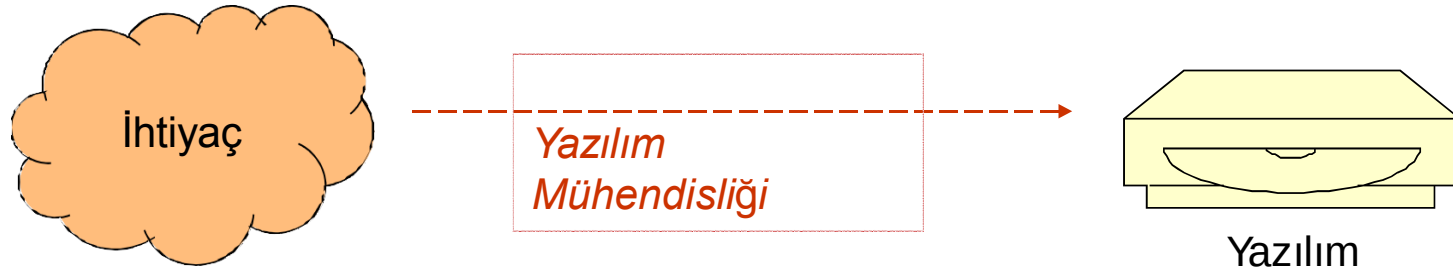
Kestirim (C Satır)	Önce	Zamanında	Gecikme	İptal
13.000	6.06%	74.77%	11.83%	7.33%
130.000	1.24%	60.76%	17.67%	20.33%
1.300.000	0.14%	28.03%	23.83%	48.00%
13.000.000	0.0%	13.67%	21.33%	65.00%

Referans:
“Patterns of
Software
Failure and
Success”,
C. Jones

Büyük kapsamlı yazılım ürünleri için, mühendislik yaklaşımı zorunlu hale gelmiştir!

Yazılım Mühendisliği Nedir?

- Yazılım üretiminin tüm etkinliklerini kapsayan mühendislik disiplinidir.
 - ▶ Sistem tanımlama gibi erken aşamalardan sistemin kullanımı esnasındaki bakımına kadar uzanan etkinlikleri kapsar.
 - ▶ Sadece teknik etkinlikleri değil, aynı zamanda yönetim etkinliklerini de içerir.



Mühendislik Yaklaşımı

- Yazılım Mühendisliği; yazılım ürününün geliştirilmesi, işletilmesi ve bakımı için uygulanan; sistematik, disiplinli ve ölçülebilir yaklaşımdır.
[IEEE, 1990]
- Mühendislik, herhangi bir bilim alanındaki bilgi birikimini sistematik olarak pratiğe geçirmeyi hedefler; bilimi ve matematiği kullanır.
 - Yönetim parametreleri: İşlev, maliyet, zaman
 - Kalite parametreleri: Dayanıklılık, bakım kolaylığı, güvenlik, kullanım kolaylığı, vb.
- Tekrarlanabilir başarılar için mühendislik yaklaşımı şarttır.
 - Mühendislik öğretisi ile bir yöntem uygulandığında, benzer sonuçları her zaman elde etme güvenliği vardır.

Yazılım Mühendisliği Neleri Kapsar

- Yazılım Gereksinim Analizi
- Yazılım Tasarım
- Yazılım Gerçekleştirme ← *Programlama*
- Yazılım Test
- Yazılım Bakım
- Yazılım Mühendislik Yönetimi
- Yazılım Konfigürasyon Yönetimi
- Yazılım Mühendisliği Süreçleri
- Yazılım Mühendisliği Araç ve Yöntemleri
- Yazılım Kalitesi

*Yazılım
Geliştirme
Kapsamı*

[“Guide to the Software Engineering Body of Knowledge”, 2004]

CASE Nedir?

- Computer-Aided Software Engineering
- Yazılım mühendisliği işleri yapmaya yarayan yazılımlar, başka bir deyişle bir yazılım projesini yönetmeye yarayan yazılımlar
 - Zaman çizelgesi
 - İş dağıtımı
 - Süreç tasarımı

İyi Bir Yazılım Nasıl Olmalı

- Bakış açısına bağlı
- Yazılım mühendisliği açısından
 - Kullanıcı dostu (adaptasyon ve kullanım kolaylığı)
 - Windows start butonu
 - İnsanların varlık gösterebildiği projeler

Yazılım Mühendisliğinde Karşılaşılan Problemler

- İnsanla uğraşmak
- Doküman yazmak
 - Yazılım ekibine yeni katılanlar, ayrılanlar
- Yazılımın türü
 - Generic
 - Çok kapsamlı sektör araştırması
 - Kişiyeye özel
- Bir önceki sistem Donanım farklılıkları İşletim sistemi farklılıkları Etik problemler

Yazılım Süreci

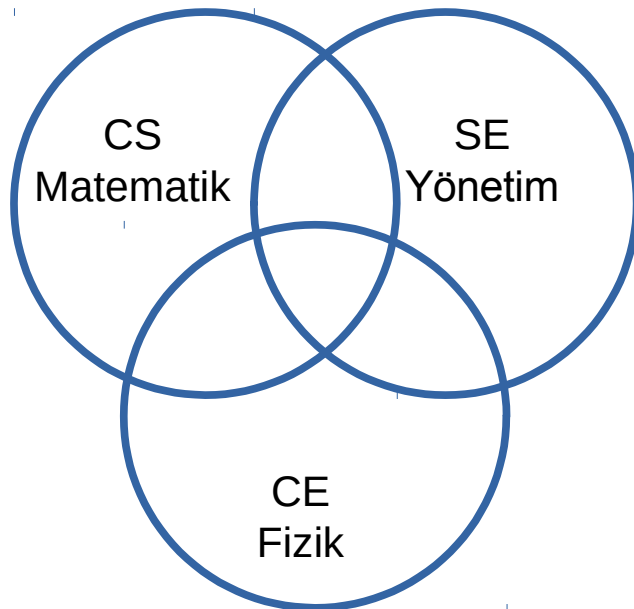
- Yazılımın oluşturulmasından, kullanım süreci ve sonrasına kadar olan zamanı kapsayan bir zaman dilimi
- Şelale Modeli (Waterfall)
 - Sistem özelliklerinin ve taleplerin belirlenmesi
 - Geliştirme
 - Geliştirilen yazılımın sistem özelliklerine ve taleplere uygunluğunun kontrolü
 - Evrim
- Modelin olumsuz yönleri
 - Bilgisayar dünyasına uygun değil
 - İstekleri ve değişimleri etkin bir şekilde yönetemez
 - Agile gibi metodolojiler bilgisayar dünyasına daha uygun

İyi Yazılım Ne Demektir

Yazılım ürünü, müşterinin beklediği işlevsellik ve performansın yanı sıra, aşağıdaki özellikleri de taşımalıdır.

- Bakım-yapılabilirlik (“maintainability”)
 - Yazılım, değişen ihtiyaçlara göre değişebilir olmalıdır.
- Güvenilirlik (“dependability”)
 - Yazılım, güvenilir olmalıdır.
- Etkinlik (“efficiency”)
 - Yazılım, sistem kaynaklarının israfına sebep olmamalıdır.
- Kabul-edilebilirlik (“acceptability”)
 - Yazılım, kullanıcıları tarafından kabul edilebilmelidir.
(Diğer bir deyişle; anlaşılabilir ve kullanılabilir olmalı, diğer sistemlerle uyumlu çalışabilmelidir.)

- Yazılım nasıl daha iyi olabilir? Farklı bakış açıları var.



Bilgisayar Bilimleri ve Yazılım Mühendisliği Arasındaki İlişki

- Bilgisayar Bilimleri; bilgisayar ve yazılım sistemlerinin temelinde yatan teoriler ve yöntemlerle ilgilenir.

- ▶ Algoritmalar ve veri yapıları, programlama dilleri, mimari, bilimsel hesaplama, işletim sistemleri, bilgi ve veri yönetimi, grafik görüntüleme ve çoklu-ortam, bilgisayar ağları, akıllı sistemler, vb.

(Fizik bilimlerinin, Elektrik ve Elektronik Mühendisliğinin temelinde yattığı gibi)

- Yazılım Mühendisliği; yazılım üretmenin pratik problemleriyle ilgilenir.

- ▶ Bunu yaparken Bilgisayar Bilimlerinin sunduğu kavramsal altyapıyı kullanır.
- ▶ Bir yazılım mühendisinin, ilişkili uygulama alanına göre (örneğin veri-işleme, animasyon, vb.), altta yatan bilgisayar bilimine hakim olması zorunludur.

Sistem Mühendisliği ve Yazılım Mühendisliği Arasındaki İlişki

- Sistem Mühendisliği; yazılımın önemli rol oynadığı, karmaşık bilgisayar sistemlerinin geliştirilmesi ve idamesi ile ilgilenir.
 - ▶ Örnek: ATM sistemi
 - ▶ Sistem Mühendisliği; sistemin genel çatısıyla ilgilidir.
 - Sistemin kullanılacağı alana ilişkin süreçlerin analiz edilmesi, sistem gereksinimlerinin tanımlanması, sistem mimarisinin oluşturulması, sistem bileşenlerinin tümleştirilmesi gibi aşamaları içerir.
 - ▶ Söz konusu bilgisayar sistemi; donanım, ağ, yazılım bileşenlerinden oluşur. Sistem Mühendisliği, bileşenlere ilişkin mühendislik etkinliklerinin detaylarıyla pek ilgilenmez.
 - Bu tür sistemler için Yazılım Mühendisliği (kapsamındaki tüm etkinliklerle birlikte), Sistem Mühendisliği altında ve onun bir parçası olarak uygulanır.
 - ▶ Sistem Mühendisliği, Yazılım Mühendisliğine kıyasla çok daha eski bir mühendislik disiplindir.

Donanım Mühendisliği ve Yazılım Mühendisliği Arasındaki İlişki

- Donanım mühendislikleri ile yazılım mühendisliğinin en belirgin farkı ürünlerindedir.
 - ▶ Yazılım ürünü diğer mühendislik ürünlerine oranla daha soyuttur.
 - ▶ Yazılım projesi geliştirme ile sonlanırken donanım projelerinde ek olarak imalat safhası vardır.
 - ▶ Seri üretim, yazılım geliştirme içerisinde neredeyse hiçten ibarettir.
 - ▶ Donanım ürünleri kullanıldıkça aşınır; yazılım ürünlerinde ise aşınma olmaz. Yalnızca baştan beri gizli bulunan hatalar, yazılım kullanıldıkça ortaya çıkar.
- Donanım mühendisliklerinde maliyet odağı seri üretim ve yıpranmayken, yazılım mühendisliğinde maliyet odağı geliştirmedir.

Yazılım Mühendisliğinde Maliyetlerin Dağılımı Nedir

- Kabaca söylersek, yazılım maliyetinin;
 - %60 : Geliştirme maliyeti,
 - %40 : Test maliyetidir (test maliyetine bulunan hataları düzeltmenin maliyeti de dahildir.)
- Müşteriye özel üretilen yazılımlar için idame (bakım) maliyeti, geliştirme maliyetinin birkaç katına çıkmaktadır.

Yazılım Mhendisliğinde Hedefler Zorluklar

- Heterojenlik (“heterogeneity”)
 - ▶ Yazılım geliştirme için, heterojen platformları ve çalıştırma ortamlarını destekleyecek teknikler geliştirmek
- Teslim (“delivery”)
 - ▶ Yazılımın zamanında teslimi için teknikler geliştirmek
- Güven (“trust”)
 - ▶ Yazılımın kullanıcıları tarafından güvenilebileceğini gösteren teknikler geliştirmek

Yazılım Mühendisliğinde Profesyonel ve Etik Gereklilik

- Yazılım mühendisliği, teknik yetkinliğin uygulanmasının yanında, aşağıdaki konularda sorumluluk gerektirir:
 - ▶ Gizlilik
 - Örnek: Çalışanların ve müşterilerin gizlilik haklarına saygı göstermek
 - ▶ Rekabet
 - Örnek: Kendi yetkinliğiniz dışında iş kabul etmemek
 - ▶ Özlük hakları
 - Örnek: Patent, mülkiyet, vb. haklarına dikkat etmek
 - ▶ Bilgisayarın amaç-dışı kullanımı
 - Örnek: Başkasına ait bir makinede oyun oynayarak virüs bulaştırmamak

Bir Meslek Olarak Yazılım Mühendisliği

- Sektörde çalışanların yaklaşık yarısı “Bilgisayar Mühendisliği” lisans eğitimine sahip
 - ▶ Alana göre farklılık gösteren lisans dereceleri var
 - Örneğin; Yönetim Bilgi Sistemleri uygulamaları için İşletme lisansı, gömülü uygulamalar için Elektrik ve Elektronik Mühendisliği lisansı
- Yazılım mühendisliği geniş kesimlerce bir meslek olarak algılanmakta
- Üniversitelerin Yazılım Mühendisliği lisans ve yüksek lisans programları var

Kaynaklar

- ☞ Software Engineering, *Ian Sommerville*, 9th edition, Addison-Wesley
- ☞ <http://web.firat.edu.tr/mbaykara/yazilimmuhendisligi.html>